API Integration

All API calls go through a single Axios instance with JWT injection and token-refresh logic. Service modules group related endpoints into typed functions.

Table of Contents

---

1. Axios Instance

src/api/axios.ts

```ts
import axios, {
  AxiosInstance,
  AxiosError,
  InternalAxiosRequestConfig,
  AxiosResponse,
} from 'axios';
import * as SecureStore from 'expo-secure-store';
import { router } from 'expo-router';
import { ENV } from '@/lib/constants';

const TOKEN_KEY = 'bsecure_jwt';
const REFRESH_KEY = 'bsecure_refresh';

let isRefreshing = false;
let failedQueue: Array<{
  resolve: (token: string) => void;
```

```
  reject: (err: unknown) => void;
}> = [];

const processQueue = (error: unknown, token: string | null) => {
  failedQueue.forEach(({ resolve, reject }) => {
    if (token) resolve(token);
    else reject(error);
  });
  failedQueue = [];
};

export const apiClient: AxiosInstance = axios.create({
  baseURL: ENV.API_BASE_URL,
  timeout: 15_000,
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json',
  },
});

// ── Request interceptor ──────────────────────────────────────

apiClient.interceptors.request.use(
  async (config: InternalAxiosRequestConfig) => {
    const token = await SecureStore.getItemAsync(TOKEN_KEY);
    if (token && config.headers) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error),
);

// ── Response interceptor ─────────────────────────────────────

apiClient.interceptors.response.use(
  (response: AxiosResponse) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as
        InternalAxiosRequestConfig & {
      _retry?: boolean;
    };

    if (error.response?.status === 401 && !
        originalRequest._retry) {
      if (isRefreshing) {
        return new Promise((resolve, reject) => {
          failedQueue.push({ resolve, reject });
        }).then((token) => {
          if (originalRequest.headers) {
            originalRequest.headers.Authorization = `Bearer $
        {token}`;
          }
```

```
          return apiClient(originalRequest);
        });
      }

      originalRequest._retry = true;
      isRefreshing = true;

      try {
        const refreshToken = await
        SecureStore.getItemAsync(REFRESH_KEY);
        if (!refreshToken) throw new Error('No refresh token');

        const { data } = await axios.post<{ access: string }>(
          `${ENV.API_BASE_URL}/auth/token/refresh/`,
          { refresh: refreshToken },
        );

        await SecureStore.setItemAsync(TOKEN_KEY, data.access);
        processQueue(null, data.access);

        if (originalRequest.headers) {
          originalRequest.headers.Authorization = `Bearer $
        {data.access}`;
        }
        return apiClient(originalRequest);
      } catch (refreshError) {
        processQueue(refreshError, null);
        await Promise.all([
          SecureStore.deleteItemAsync(TOKEN_KEY),
          SecureStore.deleteItemAsync(REFRESH_KEY),
        ]);
        router.replace('/(auth)/login');
        return Promise.reject(refreshError);
      } finally {
        isRefreshing = false;
      }
    }

    // Wrap in ApiError before rejecting
    return Promise.reject(ApiError.fromAxios(error));
  },
);

export default apiClient;
```

---

2. Auth Service

src/api/authService.ts

```ts
import apiClient from './axios';
import type {
  OTPRequestPayload,
  OTPVerifyPayload,
  AuthResponse,
} from '@/types';

export const authService = {
  /**
   * Sends an OTP to the given phone number.
   */
  requestOTP: async (payload: OTPRequestPayload): Promise<void>
        => {
    await apiClient.post('/auth/otp/request/', payload);
  },

  /**
   * Verifies the OTP and returns JWT tokens + user.
   */
  verifyOTP: async (payload: OTPVerifyPayload):
        Promise<AuthResponse> => {
    const { data } = await apiClient.post<AuthResponse>(
      '/auth/otp/verify/',
      payload,
    );
    return data;
  },

  /**
   * Refreshes the access token using the refresh token.
   */
  refreshToken: async (refresh: string): Promise<{ access:
        string }> => {
    const { data } = await apiClient.post<{ access: string }>(
      '/auth/token/refresh/',
      { refresh },
    );
    return data;
  },
};
```

---

3. Guard Service

src/api/guardService.ts

```ts
import apiClient from './axios';
import type { Guard, PaginatedResponse } from '@/types';

interface NearbyGuardsParams {
  latitude: number;
  longitude: number;
```

```
  radius: number;
}

export const guardService = {
  /**
   * Returns guards within `radius` metres of the given
       coordinates.
   */
  getNearbyGuards: async (params: NearbyGuardsParams):
       Promise<Guard[]> => {
    const { data } = await apiClient.get<Guard[]>('/guards/
       nearby/', {
      params,
    });
    return data;
  },

  /**
   * Full guard profile including skills and bio.
   */
  getGuardProfile: async (guardId: number): Promise<Guard> => {
    const { data } = await apiClient.get<Guard>(`/guards/$
       {guardId}/`);
    return data;
  },

  /**
   * Paginated reviews for a guard.
   */
  getGuardReviews: async (
    guardId: number,
    page: number = 1,
  ): Promise<PaginatedResponse<import('@/types').Review>> => {
    const { data } = await apiClient.get(
      `/guards/${guardId}/reviews/`,
      { params: { page } },
    );
    return data;
  },
};
```

---

4. Booking Service

src/api/bookingService.ts

```
import apiClient from './axios';
import type {
  Booking,
  CreateBookingPayload,
  PaginatedResponse,
} from '@/types';
```

```ts
interface BookingListParams {
  tab: 'active' | 'history';
  page?: number;
}

export const bookingService = {
  /**
   * Creates a new booking.
   */
  createBooking: async (payload: CreateBookingPayload):
        Promise<Booking> => {
    const { data } = await apiClient.post<Booking>('/bookings/',
        payload);
    return data;
  },

  /**
   * Fetches a single booking by ID.
   */
  getBooking: async (bookingId: number): Promise<Booking> => {
    const { data } = await apiClient.get<Booking>(`/bookings/$
        {bookingId}/`);
    return data;
  },

  /**
   * Paginated booking list. tab=active returns in-progress,
        tab=history returns
   * completed/cancelled.
   */
  getBookingList: async (
    params: BookingListParams,
  ): Promise<PaginatedResponse<Booking>> => {
    const { data } = await
        apiClient.get<PaginatedResponse<Booking>>(
      '/bookings/',
      { params },
    );
    return data;
  },

  /**
   * Cancels a pending or assigned booking.
   */
  cancelBooking: async (bookingId: number): Promise<Booking> => {
    const { data } = await apiClient.post<Booking>(
      `/bookings/${bookingId}/cancel/`,
    );
    return data;
  },

  /**
```

```
   * Ends an active booking and triggers billing.
   */
  endBooking: async (bookingId: number): Promise<Booking> => {
    const { data } = await apiClient.post<Booking>(
      `/bookings/${bookingId}/end/`,
    );
    return data;
  },
};
```

---

5. Wallet Service

src/api/walletService.ts

```
import apiClient from './axios';
import type {
  Wallet,
  Transaction,
  PaginatedResponse,
  TopUpPayload,
  TopUpResponse,
  ConfirmTopUpPayload,
} from '@/types';

interface TransactionListParams {
  page?: number;
  pageSize?: number;
}

export const walletService = {
  /**
   * Returns the authenticated user's wallet balance.
   */
  getWallet: async (): Promise<Wallet> => {
    const { data } = await apiClient.get<Wallet>('/wallet/');
    return data;
  },

  /**
   * Paginated transaction history.
   */
  getTransactions: async (
    params: TransactionListParams = {},
  ): Promise<PaginatedResponse<Transaction>> => {
    const { data } = await
        apiClient.get<PaginatedResponse<Transaction>>(
      '/wallet/transactions/',
      { params: { page: params.page ?? 1, page_size:
        params.pageSize ?? 20 } },
    );
```

```
    return data;
  },

  /**
   * Creates a Razorpay order for the given amount.
   * Returns order details needed to open the Razorpay checkout.
   */
  initiateTopUp: async (payload: TopUpPayload):
        Promise<TopUpResponse> => {
    const { data } = await apiClient.post<TopUpResponse>(
      '/wallet/topup/initiate/',
      payload,
    );
    return data;
  },

  /**
   * Confirms the payment after Razorpay success and credits the
        wallet.
   */
  confirmTopUp: async (payload: ConfirmTopUpPayload):
        Promise<Wallet> => {
    const { data } = await apiClient.post<Wallet>(
      '/wallet/topup/confirm/',
      payload,
    );
    return data;
  },
};
```

---

6. Review Service

src/api/reviewService.ts

```
import apiClient from './axios';
import type { Review, SubmitReviewPayload } from '@/types';

export const reviewService = {
  /**
   * Submits a star rating + comment for a completed booking.
   */
  submitReview: async (payload: SubmitReviewPayload):
        Promise<Review> => {
    const { data } = await apiClient.post<Review>('/reviews/',
        payload);
    return data;
  },
};
```

---

7. User Service

src/api/userService.ts

```ts
import apiClient from './axios';
import type { User } from '@/types';

interface UpdateProfilePayload {
  name?: string;
  email?: string;
}

export const userService = {
  /**
   * Returns the authenticated user's profile.
   */
  getProfile: async (): Promise<User> => {
    const { data } = await apiClient.get<User>('/users/me/');
    return data;
  },

  /**
   * Updates name and/or email.
   */
  updateProfile: async (payload: UpdateProfilePayload):
        Promise<User> => {
    const { data } = await apiClient.patch<User>('/users/me/',
        payload);
    return data;
  },

  /**
   * Uploads a profile photo.
   * `uri` is the local file URI returned by the image picker.
   */
  uploadProfilePhoto: async (uri: string): Promise<User> => {
    const formData = new FormData();
    formData.append('profile_photo', {
      uri,
      name: 'profile.jpg',
      type: 'image/jpeg',
    } as any);

    const { data } = await apiClient.post<User>(
      '/users/me/photo/',
      formData,
      { headers: { 'Content-Type': 'multipart/form-data' } },
    );
    return data;
  },
};
```

---

8. SOS Service

src/api/sosService.ts

```ts
import apiClient from './axios';
import type { SOSTriggerPayload } from '@/types';

interface SOSResponse {
  id: number;
  bookingId: number;
  status: 'triggered';
  message: string;
  triggeredAt: string;
}

export const sosService = {
  /**
   * Sends an SOS alert for an active booking.
   * Notifies backend, guard, and emergency response team.
   */
  triggerSOS: async (payload: SOSTriggerPayload):
        Promise<SOSResponse> => {
    const { data } = await apiClient.post<SOSResponse>(
      '/sos/trigger/',
      payload,
    );
    return data;
  },
};
```

---

9. Notification Service

src/api/notificationService.ts

```ts
import apiClient from './axios';
import type { PaginatedResponse } from '@/types';

interface AppNotification {
  id: number;
  title: string;
  body: string;
  data: Record<string, unknown>;
  isRead: boolean;
  createdAt: string;
}

export const notificationService = {
  /**
   * Sends the Expo push token to the backend so the server can
```

```
   * send targeted push notifications to this device.
   */
  registerPushToken: async (expoPushToken: string): Promise<void>
        => {
    await apiClient.post('/notifications/token/', {
      token: expoPushToken,
      platform: 'expo',
    });
  },

  /**
   * Returns paginated in-app notifications for the user.
   */
  getNotifications: async (
    page: number = 1,
  ): Promise<PaginatedResponse<AppNotification>> => {
    const { data } = await
        apiClient.get<PaginatedResponse<AppNotification>>(
      '/notifications/',
      { params: { page } },
    );
    return data;
  },

  /**
   * Marks a single notification as read.
   */
  markRead: async (notificationId: number): Promise<void> => {
    await apiClient.patch(`/notifications/${notificationId}/read/
        `);
  },

  /**
   * Marks all notifications as read.
   */
  markAllRead: async (): Promise<void> => {
    await apiClient.post('/notifications/mark-all-read/');
  },
};
```

---

10. Request / Response Interfaces

All types are centralised in `src/types/index.ts`. Here is a summary of the shapes used in API calls:

```
// ── Auth ──────────────────────────────────────────────────────

interface OTPRequestPayload {
  phone: string;          // "9876543210"
  countryCode: string;    // "+91"
```

```ts
}

interface OTPVerifyPayload {
  phone: string;
  countryCode: string;
  otp: string;            // "123456"
}

interface AuthResponse {
  access: string;         // short-lived JWT
  refresh: string;        // long-lived refresh token
  user: User;
}

// ── Guards ──────────────────────────────────────────────

interface NearbyGuardsParams {
  latitude: number;
  longitude: number;
  radius: number;         // metres
}

// ── Bookings ────────────────────────────────────────────

interface CreateBookingPayload {
  guardId: number;
  type: 'on_demand' | 'scheduled';
  pickupAddress: {
    address: string;
    latitude: number;
    longitude: number;
    landmark?: string;
  };
  scheduledAt?: string;   // ISO 8601, required for type=scheduled
  notes?: string;
}

// ── Wallet ──────────────────────────────────────────────

interface TopUpPayload {
  amount: number;         // INR, no paise
}

interface TopUpResponse {
  orderId: string;        // Razorpay order ID
  amount: number;
  currency: string;       // "INR"
  keyId: string;
}
```

```ts
interface ConfirmTopUpPayload {
  orderId: string;
  paymentId: string;
  signature: string;
}

// ── Reviews ──────────────────────────────────────────────────────────

interface SubmitReviewPayload {
  bookingId: number;
  rating: number;          // 1–5
  comment: string;
}

// ── SOS ──────────────────────────────────────────────────────────────

interface SOSTriggerPayload {
  bookingId: number;
  latitude: number;
  longitude: number;
}
```

---

11. ApiError Class

src/api/ApiError.ts

```ts
import type { AxiosError } from 'axios';

interface DjangoErrorResponse {
  detail?: string;
  non_field_errors?: string[];
  [field: string]: string | string[] | undefined;
}

const HTTP_STATUS_MESSAGES: Record<number, string> = {
  400: 'Invalid request. Please check your input.',
  401: 'Session expired. Please log in again.',
  403: 'You do not have permission to perform this action.',
  404: 'The requested resource was not found.',
  409: 'This action conflicts with the current state.',
  422: 'Validation failed. Please review the form.',
  429: 'Too many requests. Please wait a moment and try again.',
  500: 'Server error. Please try again later.',
  502: 'Service temporarily unavailable.',
  503: 'Service temporarily unavailable.',
};

export class ApiError extends Error {
```

```
public readonly status: number;
public readonly code: string;
public readonly fieldErrors: Record<string, string[]>;
public readonly rawResponse: unknown;

constructor(
  message: string,
  status: number,
  code: string = 'unknown',
  fieldErrors: Record<string, string[]> = {},
  rawResponse: unknown = null,
) {
  super(message);
  this.name = 'ApiError';
  this.status = status;
  this.code = code;
  this.fieldErrors = fieldErrors;
  this.rawResponse = rawResponse;
}

/**
 * Creates an ApiError from an Axios error.
 * Extracts the best human-readable message from Django error
      format.
 */
static fromAxios(error: AxiosError): ApiError {
  const status = error.response?.status ?? 0;
  const responseData = error.response?.data as
      DjangoErrorResponse | undefined;

  // Extract field errors
  const fieldErrors: Record<string, string[]> = {};
  if (responseData && typeof responseData === 'object') {
    Object.entries(responseData).forEach(([key, value]) => {
      if (key !== 'detail' && Array.isArray(value)) {
        fieldErrors[key] = value as string[];
      }
    });
  }

  // Extract best message
  let message =
    HTTP_STATUS_MESSAGES[status] ?? 'An unexpected error
      occurred.';

  if (responseData?.detail) {
    message = responseData.detail;
  } else if (responseData?.non_field_errors?.length) {
    message = responseData.non_field_errors[0];
  } else if (Object.keys(fieldErrors).length > 0) {
    const firstField = Object.keys(fieldErrors)[0];
    message = fieldErrors[firstField][0];
  } else if (error.code === 'ECONNABORTED') {
```

```
      message = 'Request timed out. Check your connection and try
        again.';
    } else if (!error.response) {
      message = 'Network error. Please check your internet
        connection.';
    }

    return new ApiError(
      message,
      status,
      error.code ?? 'unknown',
      fieldErrors,
      error.response?.data,
    );
  }

  /** True if error is a 4xx client error */
  get isClientError(): boolean {
    return this.status >= 400 && this.status < 500;
  }

  /** True if error is a 5xx server error */
  get isServerError(): boolean {
    return this.status >= 500;
  }

  /** True if error is a network/timeout failure (no HTTP
        response) */
  get isNetworkError(): boolean {
    return this.status === 0;
  }
}
```

Usage Example

```
import { ApiError } from '@/api/ApiError';

try {
  await bookingService.createBooking(payload);
} catch (err) {
  if (err instanceof ApiError) {
    // Show human-readable message
    Alert.alert('Booking Failed', err.message);

    // Access field-specific errors for form highlighting
    if (err.fieldErrors.pickup_address) {
      setAddressError(err.fieldErrors.pickup_address[0]);
    }
  }
}
```